# MoA Hyperparameter Tuning Pipeline

Superseded version. It uses 80 PCA components for `g-` and 300 CatBoost iterations, unlike the final pipeline (50 and 250); its output was not used by any reported run.

This notebook implements a complete hyperparameter optimization pipeline for **TabularResNet** and **CatBoostClassifier** using **Optuna** on GPU.

## 0. Setup, imports and Device Configuration

In [1]:
```
!pip install -q iterative-stratification
```

In [2]:
```
import gc
import json
import math
import os
import warnings
from catboost import CatBoostClassifier
from iterstrat.ml_stratifiers import MultilabelStratifiedKFold
import numpy as np
import optuna
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.feature_selection import VarianceThreshold
from sklearn.preprocessing import QuantileTransformer
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset

warnings.filterwarnings("ignore")
optuna.logging.set_verbosity(optuna.logging.WARNING)

# Global Constants
CLIP_EPS = 1e-15
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device in use: {DEVICE} | Visible GPUs: {torch.cuda.device_count()}")
```

```
Device in use: cuda | Visible GPUs: 2
```

# 1. Data loading and feature engineering

In [3]:
```python
# --- 1. DATA LOADING AND COMPLETE FEATURE ENGINEERING ---
def load_and_preprocess_data(data_dir="/kaggle/input/competitions/lish-moa/"):
  if not os.path.exists(data_dir):
    data_dir = ""  # Fallback to current directory

  train_features = pd.read_csv(f"{data_dir}train_features.csv")
  train_targets = pd.read_csv(f"{data_dir}train_targets_scored.csv")

  # Filter control vehicle samples (ctl_vehicle)
  non_control_mask = train_features["cp_type"] != "ctl_vehicle"
  train_features = train_features[non_control_mask].reset_index(drop=True)
  train_targets = train_targets[non_control_mask].reset_index(drop=True)

  feature_cols = [
      c for c in train_features.columns if c not in ["sig_id", "cp_type"]
  ]
  target_cols = [c for c in train_targets.columns if c != "sig_id"]

  g_cols = [c for c in feature_cols if c.startswith("g-")]
  c_cols = [c for c in feature_cols if c.startswith("c-")]

  # Advanced Feature Engineering (Row-wise statistics for g- and c-)
  df_out = train_features[
      [c for c in train_features.columns if c != "cp_type"]
  ].copy()
  df_out["g_sum"] = df_out[g_cols].sum(axis=1)
  df_out["g_mean"] = df_out[g_cols].mean(axis=1)
  df_out["g_std"] = df_out[g_cols].std(axis=1)
  df_out["g_kurtosis"] = df_out[g_cols].kurtosis(axis=1)
  df_out["g_skew"] = df_out[g_cols].skew(axis=1)

  df_out["c_sum"] = df_out[c_cols].sum(axis=1)
  df_out["c_mean"] = df_out[c_cols].mean(axis=1)
  df_out["c_std"] = df_out[c_cols].std(axis=1)
  df_out["c_kurtosis"] = df_out[c_cols].kurtosis(axis=1)
  df_out["c_skew"] = df_out[c_cols].skew(axis=1)

  dummy_cols = ["cp_time", "cp_dose"]
  df_out = pd.get_dummies(df_out, columns=dummy_cols, dtype=np.float64)

  full_feature_cols = [c for c in df_out.columns if c != "sig_id"]
  dummy_feature_cols = [
      c
```

```
      for c in full_feature_cols
      if any(c.startswith(d) for d in dummy_cols)
  ]

  X = df_out[full_feature_cols].values
  Y = train_targets[target_cols].values

  return X, Y, full_feature_cols, g_cols, c_cols, dummy_feature_cols

def compute_moa_log_loss(y_true, y_pred, eps=CLIP_EPS):
  y_true = np.asarray(y_true, dtype=np.float64)
  y_pred = np.clip(np.asarray(y_pred, dtype=np.float64), eps, 1 - eps)
  loss_matrix = -(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))
  return np.mean(loss_matrix)
```

# 2. Model architecture definitions

In [4]:

```
# --- 2. MODEL ARCHITECTURE DEFINITIONS ---

# Model 1: TabularResNet
class TabularResNet(nn.Module):

  name = "resnet"

  def __init__(
      self, input_dim, output_dim=206, hidden_dim=512, dropout_rate=0.2
  ):
    super().__init__()
    self.in_proj = nn.Sequential(
        nn.BatchNorm1d(input_dim),
        nn.Dropout(dropout_rate),
        nn.Linear(input_dim, hidden_dim),
        nn.BatchNorm1d(hidden_dim),
        nn.SiLU(),
    )
    self.block1 = self._make_block(hidden_dim, dropout_rate)
    self.block2 = self._make_block(hidden_dim, dropout_rate)
    self.out_proj = nn.Sequential(
        nn.SiLU(), nn.Dropout(dropout_rate), nn.Linear(hidden_dim, output_dim)
    )

  @staticmethod
  def _make_block(hidden_dim, dropout_rate):
    return nn.Sequential(
        nn.Dropout(dropout_rate),
        nn.Linear(hidden_dim, hidden_dim),
```

```
        nn.BatchNorm1d(hidden_dim),
        nn.SiLU(),
        nn.Dropout(dropout_rate),
        nn.Linear(hidden_dim, hidden_dim),
        nn.BatchNorm1d(hidden_dim),
    )

  def forward(self, x):
    x = self.in_proj(x)
    x = F.silu(x + self.block1(x))
    x = F.silu(x + self.block2(x))
    return self.out_proj(x)


# Model 2: FT-Transformer
class FeatureTokenizer(nn.Module):

  def __init__(self, num_features, d_token):
    super().__init__()
    self.weight = nn.Parameter(torch.empty(num_features, d_token))
    self.bias = nn.Parameter(torch.empty(num_features, d_token))
    nn.init.kaiming_uniform_(self.weight, a=math.sqrt(5))
    nn.init.zeros_(self.bias)

  def forward(self, x):
    return x.unsqueeze(-1) * self.weight + self.bias


class FTTransformer(nn.Module):

  name = "ft_transformer"

  def __init__(
      self,
      input_dim,
      output_dim=206,
      d_token=64,
      n_blocks=2,
      attention_heads=4,
      dropout=0.2,
  ):
    super().__init__()
    self.tokenizer = FeatureTokenizer(input_dim, d_token)
    self.cls_token = nn.Parameter(torch.zeros(1, 1, d_token))
    nn.init.normal_(self.cls_token, std=0.02)

    encoder_layer = nn.TransformerEncoderLayer(
        d_model=d_token,
```

```
        nhead=attention_heads,
        dim_feedforward=d_token * 4,
        dropout=dropout,
        activation="gelu",
        batch_first=True,
    )
    self.transformer = nn.TransformerEncoder(
        encoder_layer, num_layers=n_blocks, enable_nested_tensor=False
    )
    self.layer_norm = nn.LayerNorm(d_token)
    self.head = nn.Sequential(nn.Dropout(dropout), nn.Linear(d_token, output_dim))

  def forward(self, x):
    tokens = self.tokenizer(x)
    cls_tokens = self.cls_token.expand(x.size(0), -1, -1)
    x = torch.cat((cls_tokens, tokens), dim=1)
    x = self.transformer(x)
    cls_out = self.layer_norm(x[:, 0, :])
    return self.head(cls_out)
```

# 3. Precomputing feature transformations per fold

In [5]:

```
# --- 3. PRECOMPUTING FEATURE TRANSFORMATIONS PER FOLD ---

def precompute_pca_folds(
    X,
    Y,
    feature_cols,
    g_cols,
    c_cols,
    dummy_cols,
    n_splits=5,
    pca_g_dim=80,
    pca_c_dim=15,
):
  print(f"Precomputing full PCA Pipeline across {n_splits} Folds...")
  mskf = MultilabelStratifiedKFold(
      n_splits=n_splits, shuffle=True, random_state=42
  )
  cached_folds = []

  for train_idx, val_idx in mskf.split(X, Y):
    X_tr, Y_tr = X[train_idx], Y[train_idx]
    X_va, Y_va = X[val_idx], Y[val_idx]

    selector = VarianceThreshold(threshold=0.0)
```

```
    X_tr_vt = selector.fit_transform(X_tr)
    X_va_vt = selector.transform(X_va)
    support = selector.get_support()
    selected_cols = [c for c, keep in zip(feature_cols, support) if keep]

    dummy_idx = [i for i, c in enumerate(selected_cols) if c in dummy_cols]
    cont_idx = [i for i, c in enumerate(selected_cols) if c not in dummy_cols]

    X_tr_cont, X_va_cont = X_tr_vt[:, cont_idx], X_va_vt[:, cont_idx]
    X_tr_dummy, X_va_dummy = X_tr_vt[:, dummy_idx], X_va_vt[:, dummy_idx]
    cont_cols = [selected_cols[i] for i in cont_idx]

    g_idx_in_cont = [i for i, c in enumerate(cont_cols) if c in g_cols]
    c_idx_in_cont = [i for i, c in enumerate(cont_cols) if c in c_cols]

    qt = QuantileTransformer(
        n_quantiles=100, output_distribution="normal", random_state=42
    )
    X_tr_tf = qt.fit_transform(X_tr_cont)
    X_va_tf = qt.transform(X_va_cont)

    pca_g = PCA(n_components=pca_g_dim, random_state=42)
    tr_pca_g = pca_g.fit_transform(X_tr_tf[:, g_idx_in_cont])
    va_pca_g = pca_g.transform(X_va_tf[:, g_idx_in_cont])

    pca_c = PCA(n_components=pca_c_dim, random_state=42)
    tr_pca_c = pca_c.fit_transform(X_tr_tf[:, c_idx_in_cont])
    va_pca_c = pca_c.transform(X_va_tf[:, c_idx_in_cont])

    # Full concatenation as in the main training pipeline
    X_tr_final = np.hstack([X_tr_tf, tr_pca_g, tr_pca_c, X_tr_dummy])
    X_va_final = np.hstack([X_va_tf, va_pca_g, va_pca_c, X_va_dummy])

    cached_folds.append({
        "X_tr": X_tr_final,
        "Y_tr": Y_tr,
        "X_va": X_va_final,
        "Y_va": Y_va,
    })

  return cached_folds


def precompute_raw_folds(X, Y, feature_cols, dummy_cols, n_splits=5):
  print(f"Precomputing Raw Pipeline without PCA across {n_splits} Folds...")
  mskf = MultilabelStratifiedKFold(
      n_splits=n_splits, shuffle=True, random_state=42
  )
```

```
  cached_folds = []

  for train_idx, val_idx in mskf.split(X, Y):
    X_tr, Y_tr = X[train_idx], Y[train_idx]
    X_va, Y_va = X[val_idx], Y[val_idx]

    selector = VarianceThreshold(threshold=0.0)
    X_tr_vt = selector.fit_transform(X_tr)
    X_va_vt = selector.transform(X_va)
    support = selector.get_support()
    selected_cols = [c for c, keep in zip(feature_cols, support) if keep]

    dummy_idx = [i for i, c in enumerate(selected_cols) if c in dummy_cols]
    cont_idx = [i for i, c in enumerate(selected_cols) if c not in dummy_cols]

    X_tr_cont, X_va_cont = X_tr_vt[:, cont_idx], X_va_vt[:, cont_idx]
    X_tr_dummy, X_va_dummy = X_tr_vt[:, dummy_idx], X_va_vt[:, dummy_idx]

    qt = QuantileTransformer(
        n_quantiles=100, output_distribution="normal", random_state=42
    )
    X_tr_tf = qt.fit_transform(X_tr_cont)
    X_va_tf = qt.transform(X_va_cont)

    X_tr_final = np.hstack([X_tr_tf, X_tr_dummy])
    X_va_final = np.hstack([X_va_tf, X_va_dummy])

    cached_folds.append({
        "X_tr": X_tr_final,
        "Y_tr": Y_tr,
        "X_va": X_va_final,
        "Y_va": Y_va,
    })

  return cached_folds
```

# 4. Optuna objective functions for all 3 models

In [6]:

```
# --- 4. OPTUNA OBJECTIVE FUNCTIONS FOR ALL 3 MODELS ---

# 1. OBJECTIVE: TABULAR RESNET
def create_objective_resnet(cached_folds):

  def objective(trial):
    num_gpus = torch.cuda.device_count()
    gpu_id = trial.number % num_gpus if num_gpus > 0 else 0
```

```
    device = torch.device(f"cuda:{gpu_id}" if num_gpus > 0 else "cpu")

    lr = trial.suggest_float("lr", 1e-4, 2e-3, log=True)
    dropout = trial.suggest_float("dropout", 0.1, 0.35)
    weight_decay = trial.suggest_float("weight_decay", 1e-6, 1e-3, log=True)
    hidden_dim = trial.suggest_categorical("hidden_dim", [256, 512, 768])
    batch_size = trial.suggest_categorical("batch_size", [128, 256])

    scores = []
    try:
      for fold in cached_folds:
        X_tr, Y_tr = fold["X_tr"], fold["Y_tr"]
        X_va, Y_va = fold["X_va"], fold["Y_va"]

        train_ds = TensorDataset(
            torch.tensor(X_tr, dtype=torch.float32),
            torch.tensor(Y_tr, dtype=torch.float32),
        )
        train_loader = DataLoader(
            train_ds, batch_size=batch_size, shuffle=True, drop_last=True
        )

        model = TabularResNet(
            X_tr.shape[1],
            Y_tr.shape[1],
            hidden_dim=hidden_dim,
            dropout_rate=dropout,
        ).to(device)
        optimizer = torch.optim.AdamW(
            model.parameters(), lr=lr, weight_decay=weight_decay
        )
        scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
            optimizer, mode="min", factor=0.1, patience=3
        )
        criterion = nn.BCEWithLogitsLoss()
        scaler = (
            torch.amp.GradScaler("cuda") if device.type == "cuda" else None
        )

        best_val_loss = float("inf")
        patience, steps_no_improve = 8, 0

        for epoch in range(35):
          model.train()
          for bx, by in train_loader:
            bx, by = bx.to(device, non_blocking=True), by.to(
                device, non_blocking=True
            )
```

```
            by_smooth = by * (1.0 - 0.001) + 0.5 * 0.001
            optimizer.zero_grad()

            if device.type == "cuda":
              with torch.amp.autocast("cuda"):
                out = model(bx)
                loss = criterion(out, by_smooth)
              if torch.isnan(loss):
                raise optuna.TrialPruned()
              scaler.scale(loss).backward()
              scaler.unscale_(optimizer)
              torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
              scaler.step(optimizer)
              scaler.update()
            else:
              out = model(bx)
              loss = criterion(out, by_smooth)
              loss.backward()
              optimizer.step()

          model.eval()
          with torch.no_grad():
            val_tensor = torch.tensor(
                X_va, dtype=torch.float32, device=device
            )
            if device.type == "cuda":
              with torch.amp.autocast("cuda"):
                val_logits = model(val_tensor)
            else:
              val_logits = model(val_tensor)
            val_preds = torch.sigmoid(val_logits.float()).cpu().numpy()

          val_loss = compute_moa_log_loss(Y_va, val_preds)
          scheduler.step(val_loss)

          if val_loss < best_val_loss:
            best_val_loss = val_loss
            steps_no_improve = 0
          else:
            steps_no_improve += 1

          if steps_no_improve >= patience:
            break

        scores.append(best_val_loss)

      return np.mean(scores)
    except Exception:
```

```
      raise optuna.TrialPruned()
    finally:
      if torch.cuda.is_available():
        torch.cuda.empty_cache()
      gc.collect()

  return objective


# 2. OBJECTIVE: FT-TRANSFORMER
def create_objective_fttransformer(cached_raw_folds):

  def objective(trial):
    num_gpus = torch.cuda.device_count()
    gpu_id = trial.number % num_gpus if num_gpus > 0 else 0
    device = torch.device(f"cuda:{gpu_id}" if num_gpus > 0 else "cpu")

    lr = trial.suggest_float("lr", 1e-4, 1e-3, log=True)
    dropout = trial.suggest_float("dropout", 0.05, 0.25)
    weight_decay = trial.suggest_float("weight_decay", 1e-6, 1e-3, log=True)
    d_token = trial.suggest_categorical("d_token", [16, 32, 64])
    n_blocks = trial.suggest_int("n_blocks", 1, 2)
    attention_heads = trial.suggest_categorical("attention_heads", [2, 4])
    batch_size = trial.suggest_categorical("batch_size", [128, 256])

    scores = []
    try:
      for fold in cached_raw_folds:
        X_tr, Y_tr = fold["X_tr"], fold["Y_tr"]
        X_va, Y_va = fold["X_va"], fold["Y_va"]

        train_ds = TensorDataset(
            torch.tensor(X_tr, dtype=torch.float32),
            torch.tensor(Y_tr, dtype=torch.float32),
        )
        train_loader = DataLoader(
            train_ds, batch_size=batch_size, shuffle=True, drop_last=True
        )
        val_loader = DataLoader(
            TensorDataset(torch.tensor(X_va, dtype=torch.float32)),
            batch_size=128,
            shuffle=False,
        )

        model = FTTransformer(
            X_tr.shape[1],
            Y_tr.shape[1],
            d_token=d_token,
```

```
            n_blocks=n_blocks,
            attention_heads=attention_heads,
            dropout=dropout,
        ).to(device)
        optimizer = torch.optim.AdamW(
            model.parameters(), lr=lr, weight_decay=weight_decay
        )
        scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
            optimizer, mode="min", factor=0.1, patience=3
        )
        criterion = nn.BCEWithLogitsLoss()
        scaler = (
            torch.amp.GradScaler("cuda") if device.type == "cuda" else None
        )

        best_val_loss = float("inf")
        patience, steps_no_improve = 6, 0

        for epoch in range(25):
          model.train()
          for bx, by in train_loader:
            bx, by = bx.to(device, non_blocking=True), by.to(
                device, non_blocking=True
            )
            by_smooth = by * (1.0 - 0.001) + 0.5 * 0.001
            optimizer.zero_grad()

            if device.type == "cuda":
              with torch.amp.autocast("cuda"):
                out = model(bx)
                loss = criterion(out, by_smooth)
              if torch.isnan(loss):
                raise optuna.TrialPruned()
              scaler.scale(loss).backward()
              scaler.unscale_(optimizer)
              torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
              scaler.step(optimizer)
              scaler.update()
            else:
              out = model(bx)
              loss = criterion(out, by_smooth)
              loss.backward()
              optimizer.step()

          model.eval()
          val_preds_list = []
          with torch.no_grad():
            for (bx_val,) in val_loader:
```

```
            bx_val = bx_val.to(device, non_blocking=True)
            if device.type == "cuda":
              with torch.amp.autocast("cuda"):
                logits = model(bx_val)
            else:
              logits = model(bx_val)
            val_preds_list.append(
                torch.sigmoid(logits.float()).cpu().numpy()
            )
        val_preds = np.vstack(val_preds_list)
        val_loss = compute_moa_log_loss(Y_va, val_preds)
        scheduler.step(val_loss)

        if val_loss < best_val_loss:
          best_val_loss = val_loss
          steps_no_improve = 0
        else:
          steps_no_improve += 1

        if steps_no_improve >= patience:
          break

      scores.append(best_val_loss)

    return np.mean(scores)
  except Exception:
    raise optuna.TrialPruned()
  finally:
    if torch.cuda.is_available():
      torch.cuda.empty_cache()
    gc.collect()

return objective


# 3. OBJECTIVE: CATBOOST CLASSIFIER
def create_objective_catboost(cached_raw_folds):

  def objective(trial):
    num_gpus = torch.cuda.device_count()
    depth = trial.suggest_int("depth", 3, 6)
    l2_leaf_reg = trial.suggest_float("l2_leaf_reg", 1.0, 10.0)
    learning_rate = trial.suggest_float("learning_rate", 0.02, 0.1, log=True)
    subsample = trial.suggest_float("subsample", 0.6, 0.9)

    scores = []
    for fold_data in cached_raw_folds:
      X_tr, Y_tr = fold_data["X_tr"], fold_data["Y_tr"]
```

```
        X_va, Y_va = fold_data["X_va"], fold_data["Y_va"]
        num_targets = Y_tr.shape[1]

        # Handling constant targets in the training fold
        col_is_constant = np.array(
            [len(np.unique(Y_tr[:, t])) < 2 for t in range(num_targets)]
        )
        active_cols = np.where(~col_is_constant)[0]
        constant_cols = np.where(col_is_constant)[0]
        constant_vals = (Y_tr.sum(axis=0) + 0.5) / (Y_tr.shape[0] + 1.0)

        try:
          cb = CatBoostClassifier(
              iterations=300,
              depth=depth,
              l2_leaf_reg=l2_leaf_reg,
              learning_rate=learning_rate,
              subsample=subsample,
              bootstrap_type="Bernoulli",
              loss_function="MultiLogloss",
              eval_metric="MultiLogloss",
              task_type="GPU" if num_gpus > 0 else "CPU",
              verbose=0,
              random_seed=42,
          )
          cb.fit(X_tr, Y_tr[:, active_cols])
          val_preds = np.zeros((X_va.shape[0], num_targets), dtype=np.float64)
          if len(active_cols) > 0:
            val_preds[:, active_cols] = cb.predict_proba(X_va)
          if len(constant_cols) > 0:
            val_preds[:, constant_cols] = constant_vals[constant_cols]
        except Exception:
          raise optuna.TrialPruned()

        fold_loss = compute_moa_log_loss(Y_va, val_preds)
        if np.isnan(fold_loss):
          raise optuna.TrialPruned()
        scores.append(fold_loss)

      return np.mean(scores)

    return objective
```

## 5. Execution pipeline and hyperparameter export

In [7]:

```python
# --- 5. EXECUTION PIPELINE AND HYPERPARAMETER EXPORT ---
def run_tuning_pipeline(
    data_dir="/kaggle/input/competitions/lish-moa/",
    output_file="best_hyperparameters.json",
    n_splits=5,
    resnet_trials=30,
    fttransformer_trials=15,
    catboost_trials=10,
    catboost_timeout=1200,
):
  print("=== STARTING HYPERPARAMETER TUNING PIPELINE (3 MODELS) ===")

  # 1. Data Loading and Feature Engineering
  print("\n[1/6] Loading and preprocessing data...")
  X, Y, feature_cols, g_cols, c_cols, dummy_cols = load_and_preprocess_data(
      data_dir
  )
  print(f"Dataset loaded -> Shape X: {X.shape}, Shape Y: {Y.shape}")

  # 2. CPU Caching of Folds
  print(f"\n[2/6] Precomputing transformed features across {n_splits} Folds...")
  cached_pca_folds = precompute_pca_folds(
      X, Y, feature_cols, g_cols, c_cols, dummy_cols, n_splits=n_splits
  )
  cached_raw_folds = precompute_raw_folds(
      X, Y, feature_cols, dummy_cols, n_splits=n_splits
  )
  print("Caching successfully completed. Data ready in RAM.")

  num_gpus = torch.cuda.device_count()
  n_jobs_gpu = 2 if num_gpus >= 2 else 1
  best_params = {}

  # 3. TabularResNet Tuning
  print("\n[3/6] Starting Optuna Tuning for TabularResNet...")
  objective_resnet = create_objective_resnet(cached_pca_folds)
  study_resnet = optuna.create_study(direction="minimize")
  study_resnet.optimize(
      objective_resnet, n_trials=resnet_trials, n_jobs=n_jobs_gpu
  )
  print(f"Best ResNet LogLoss ({n_splits}-Fold): {study_resnet.best_value:.5f}")
  best_params["ResNet"] = study_resnet.best_params

  # 4. FT-Transformer Tuning
  print("\n[4/6] Starting Optuna Tuning for FT-Transformer...")
  objective_ft = create_objective_fttransformer(cached_raw_folds)
  study_ft = optuna.create_study(direction="minimize")
```

```
  study_ft.optimize(
      objective_ft, n_trials=fttransformer_trials, n_jobs=n_jobs_gpu
  )
  print(
      "Best FT-Transformer LogLoss"
      f" ({n_splits}-Fold): {study_ft.best_value:.5f}"
  )
  best_params["FTTransformer"] = study_ft.best_params

  # 5. CatBoostClassifier Tuning
  print("\n[5/6] Starting Optuna Tuning for CatBoostClassifier...")
  objective_cb = create_objective_catboost(cached_raw_folds)
  study_cb = optuna.create_study(direction="minimize")
  study_cb.optimize(
      objective_cb, n_trials=catboost_trials, timeout=catboost_timeout
  )
  print(
      f"Best CatBoost LogLoss ({n_splits}-Fold): {study_cb.best_value:.5f}"
  )
  best_params["CatBoost"] = study_cb.best_params

  # 6. JSON Export
  print(f"\n[6/6] Exporting optimal hyperparameters to '{output_file}'...")
  with open(output_file, "w") as f:
    json.dump(best_params, f, indent=4)

  print("\n=== TUNING PIPELINE COMPLETED SUCCESSFULLY ===")
  print(f"All optimal parameters saved to '{output_file}'.")
  return best_params


# Full pipeline execution
best_hyperparams = run_tuning_pipeline(n_splits=5)
```

```
=== STARTING HYPERPARAMETER TUNING PIPELINE (3 MODELS) ===

[1/6] Loading and preprocessing data...
Dataset loaded -> Shape X: (21948, 887), Shape Y: (21948, 206)

[2/6] Precomputing transformed features across 5 Folds...
Precomputing full PCA Pipeline across 5 Folds...
Precomputing Raw Pipeline without PCA across 5 Folds...
Caching successfully completed. Data ready in RAM.

[3/6] Starting Optuna Tuning for TabularResNet...
Best ResNet LogLoss (5-Fold): 0.01602

[4/6] Starting Optuna Tuning for FT-Transformer...
Best FT-Transformer LogLoss (5-Fold): 0.01698

[5/6] Starting Optuna Tuning for CatBoostClassifier...
Best CatBoost LogLoss (5-Fold): 0.01767

[6/6] Exporting optimal hyperparameters to 'best_hyperparameters.json'...

=== TUNING PIPELINE COMPLETED SUCCESSFULLY ===
All optimal parameters saved to 'best_hyperparameters.json'.
```